

React & TypeScript Typisierung

Dokumentation der wichtigsten Typen, Event-Handler und Best Practices

1. Häufig genutzte React-Typen

Komponenten-Props

- `React.ReactNode` : Der flexibelste Typ für Kind-Elemente (`children`). Erlaubt alles, was React rendern kann: Strings, Zahlen, JSX-Elemente, Arrays davon oder `null` / `undefined` .
- `React.JSX.Element` : Wird verwendet, wenn exakt ein valides JSX-Element (z. B. `<div />`) erzwungen werden soll (keine reinen Strings oder Zahlen).

Hooks (useState & useRef)

- `useState<T>` : Für primitive Datentypen greift die automatische Typableitung. Bei komplexeren Objekten oder Arrays wird der Typ explizit übergeben:

```
const [user, setUser] = useState<User | null>(null);
const [items, setItems] = useState<string[]>([]);
```

- `React.useRef<T>` :
 - Für DOM-Elemente (z. B. Input): `const inputRef = useRef<HTMLInputElement>(null);`
 - Als veränderlicher Speicher (Mutable Value): `const countRef = useRef<number>(0);`

2. Typen für Event-Handling (z. B. onChange)

React verwendet ein eigenes System namens *SyntheticEvents* für Cross-Browser-Kompatibilität. Ein Typ setzt sich meist aus dem Event-Typ und dem HTML-Element zusammen: `React.EventTyp<HTML-Element-Typ>` .

onChange (Eingabefelder, Selects, Textareas)

Der Standardtyp für Wertänderungen ist `React.ChangeEvent<T>` .

- **Text-Inputs:** `React.ChangeEvent<HTMLInputElement>`
- **Dropdowns (<select>):** `React.ChangeEvent<HTMLSelectElement>`
- **Textbereiche (<textarea>):** `React.ChangeEvent<HTMLTextAreaElement>`

Weitere wichtige Event-Typen

Event / Methode	TypeScript / React Typ	Beschreibung
<code>onSubmit</code>	<code>React.FormEvent<HTMLFormElement></code>	Für Formulare (wichtig für <code>e.preventDefault()</code>)
<code>onClick</code>	<code>React.MouseEvent<HTMLButtonElement></code>	Für Klick-Aktionen auf Buttons, Divs oder Links
<code>onKeyDown</code> / <code>onKeyUp</code>	<code>React.KeyboardEvent<HTMLInputElement></code>	Für Tastatur-Aktionen (z. B. Abfangen der Enter-Taste)
<code>onScroll</code>	<code>React.UIEvent<HTMLDivElement></code>	Für Scroll-Aktionen (z. B. Infinite Scroll)
<code>onDragOver</code> / <code>onDrop</code>	<code>React.DragEvent<HTMLDivElement></code>	Für Drag-and-Drop-Schnittstellen

3. Die Bedeutung von `() => void` und Alternativen

Der Typ `() => void` beschreibt eine Funktion, die **keine Parameter** erwartet und **keinen Wert zurückgibt** (bzw. `undefined` liefert). Es ist der Standard für reine Callback-Aktionen.

Alternativen im Kontext (z. B. AuthContext):

- **Asynchrones Logout (API-Request):** `logout: () => Promise<void>;`
- **Mit Statusrückgabe (Erfolg/Fehlgeschlagen):** `logout: () => boolean;`
- **Mit optionalen Parametern (z. B. Redirect-Ziel):** `logout: (redirectTo?: string) => void;`

4. Was ist `React.FC` ?

`React.FC` steht für `React.FunctionComponent`. Es ist ein generischer Typ zur Definition von funktionalen Komponenten.

Moderne Empfehlung: Die React-Community und das Core-Team empfehlen heute meistens, auf `React.FC` zu verzichten und stattdessen Props direkt als normale TypeScript-Funktionsparameter zu deklarieren:

```
// Moderne und flexiblere Alternative:
interface WelcomeProps {
  name: string;
}

export function Welcome({ name }: WelcomeProps) {
  return <h1>Hallo, {name}</h1>;
}
```

5. Praxisbeispiel: Formular mit LocalStorage-Anbindung

Das folgende Beispiel implementiert die Erfassung von Benutzernamen, Geschlecht, Telefonnummer und Geburtstag inklusive Typisierung des States, des kombinierten Event-Handlers und der Persistierung.

```
import React, { useState } from 'react';

interface UserFormData {
  username: string;
  gender: 'male' | 'female' | 'other' | '';
  phone: string;
  birthday: string;
}

const LOCAL_STORAGE_KEY = 'user_profile_data';

export const UserForm: React.FC = () => {
  // Lazy Initializer lädt Daten einmalig aus dem LocalStorage
  const [formData, setFormData] = useState<UserFormData>(() => {
    const savedData = localStorage.getItem(LOCAL_STORAGE_KEY);
    return savedData ? (JSON.parse(savedData) as UserFormData) : {
      username: '', gender: '', phone: '', birthday: ''
    };
  });

  // Kombiniertes Change-Handler für verschiedene HTML-Elemente
  const handleChange = (event: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>) => {
    const { name, value } = event.target;
    setFormData(prev => ({ ...prev, [name]: value }));
  };

  const handleSubmit = (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    localStorage.setItem(LOCAL_STORAGE_KEY, JSON.stringify(formData));
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="username" value={formData.username} onChange={handleChange} required />
      <select name="gender" value={formData.gender} onChange={handleChange} required>
        <option value="" disabled>Wählen...</option>
        <option value="male">Männlich</option>
      </select>
      <input type="tel" name="phone" value={formData.phone} onChange={handleChange} />
      <input type="date" name="birthday" value={formData.birthday} onChange={handleChange} />
      <button type="submit">Speichern</button>
    </form>
  );
};
```